

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Аппаратное и программное обеспечение сетей»
Тема: «Программирование сетевых приложений на базе платформы java и библиотеки
java.net.»

Выполнил:
Студент 3 курса
Группы ИИ-28
Асташёнок Д.С.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: Цель цикла работ – изучить основы программирования сетевых приложений на базе библиотеки java.net. Приобрести навыки по практическому использованию библиотеки для реализации сетевых приложений на базе протокола UDP.

Вариант 2.7

Ход работы:

UdpClient.java:

```
import java.awt.BorderLayout;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.io.*;
import java.net.*;
import java.util.Date;
import javax.swing.*;

public class UdpClient extends JFrame {
    private static final String DEFAULT_HOST = "127.0.0.1";
    private static final int DEFAULT_PORT = UdpServer.PORT;
    private static final int BUF = 4096;

    private DatagramSocket socket;
    private InetAddress remoteAddr;
    private int remotePort;
    private boolean startedLogged;
    private final JTextArea log = new JTextArea();
    private final JTextField input = new JTextField();
    private final PrintWriter journal;

    public UdpClient() throws IOException {
        super("UDP client, var 2 / client 7");
        journal = new PrintWriter(new FileWriter("udp-client.log", true),
true);
        log.setEditable(false);
        add(new JScrollPane(log), BorderLayout.CENTER);

        JButton sendBtn = new JButton("Отправить");
        sendBtn.addActionListener(e -> sendLine());
        input.addActionListener(e -> sendLine());
        input.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_PAGE_DOWN) sendLine();
            }
        });
        JPanel south = new JPanel(new BorderLayout(4, 0));
        south.add(input, BorderLayout.CENTER);
        south.add(sendBtn, BorderLayout.EAST);
        add(south, BorderLayout.SOUTH);

        setSize(560, 400);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        addWindowListener(new java.awt.event.WindowAdapter() {
            public void windowClosing(java.awt.event.WindowEvent e) {
                closeConn();
            }
        });
    }
}
```

```

        log.append("Автоподключение к " + DEFAULT_HOST + ":" + DEFAULT_PORT
+ "\n");
        log.append("Отправка: кнопка «Отправить», PgDn ( Fn+Down )\n");
        log.append("Разрыв: disconnect " + DEFAULT_HOST + " " +
DEFAULT_PORT + "\n");
        setVisible(true);
        tryConnect();
        input.requestFocusInWindow();
    }

    private synchronized boolean tryConnect() {
        if (socket != null && !socket.isClosed()) return true;
        try {
            socket = new DatagramSocket();
            remoteAddr = InetAddress.getByName(DEFAULT_HOST);
            remotePort = DEFAULT_PORT;
            socket.connect(remoteAddr, remotePort);
            startedLogged = true;
            journal.println(new Date() + " CONNECTION START " +
DEFAULT_HOST + ":" + remotePort);
            log.append("подключен к " + DEFAULT_HOST + ":" + remotePort +
"\n");

            new Thread(this::readLoop).start();
            return true;
        } catch (Exception ex) {
            log.append("сокет не открылся: " + ex.getMessage() + "\n");
            socket = null;
            return false;
        }
    }

    private void sendLine() {
        String text = input.getText();
        input.setText("");
        if (text.isEmpty()) return;

        if (text.startsWith("disconnect")) {
            String[] hp = parseHostPort(text, "disconnect");
            if (hp == null) {
                log.append("Формат: disconnect <адрес> <порт>\n");
                return;
            }
            synchronized (this) {
                if (socket == null) {
                    log.append("соединения нет\n");
                    return;
                }
                if (!hp[0].equals(DEFAULT_HOST) || Integer.parseInt(hp[1])
!= remotePort) {
                    log.append("не совпадает с текущим " + DEFAULT_HOST +
":" + remotePort + "\n");
                    return;
                }
            }
            log.append("disconnect " + hp[0] + ":" + hp[1] + "\n");
            closeConn();
            return;
        }

        if (text.startsWith("connect ")) {
            log.append("автоподключение, команда connect не нужна\n");

```

```

        return;
    }

    synchronized (this) {
        if ((socket == null || socket.isClosed()) && !tryConnect())
return;
        try {
            byte[] data = text.getBytes();
            DatagramPacket datagram = new DatagramPacket(
                data, data.length, remoteAddr, remotePort);
            socket.send(datagram);
            journal.println(new Date() + " SENT " + text);
        } catch (IOException e) {
            log.append("отправка не удалась: " + e.getMessage() +
"\n");
            return;
        }
        log.append(">>> " + text + "\n");
    }

    private static String[] parseHostPort(String text, String cmd) {
        String rest = text.substring(cmd.length()).trim();
        if (rest.isEmpty()) return null;
        rest = rest.replace(':', ' ');
        String[] p = rest.split("\\s+");
        if (p.length != 2) return null;
        try {
            Integer.parseInt(p[1]);
        } catch (NumberFormatException e) {
            return null;
        }
        return p;
    }

    private void readLoop() {
        DatagramSocket local;
        synchronized (this) {
            local = socket;
        }
        if (local == null) return;
        byte[] buf = new byte[BUF];
        try {
            while (!local.isClosed()) {
                DatagramPacket packet = new DatagramPacket(buf,
buf.length);
                local.receive(packet);
                String msg = new String(packet.getData(), 0,
packet.getLength());
                SwingUtilities.invokeLater(() -> log.append("<<< " + msg +
"\n"));
            }
        } catch (IOException e) {
            // закрытие сокета из disconnect прерывает receive
        }
        SwingUtilities.invokeLater(() -> log.append("соединение
закрыто\n"));
        closeConn();
    }

    private synchronized void closeConn() {

```

```

        if (socket == null) return;
        if (startedLogged) {
            journal.println(new Date() + " CONNECTION END");
            startedLogged = false;
        }
        socket.close();
        socket = null;
        remoteAddr = null;
        remotePort = 0;
    }

    public static void main(String[] args) throws IOException {
        new UdpClient();
    }
}

```

UdpServer.java:

```

import java.io.*;
import java.net.*;
import java.util.HashMap;
import java.util.Map;

public class UdpServer {
    public static final int PORT = 6666;
    private static final int BUF = 4096;

    static class SaveJob {
        String name;
        StringBuilder content = new StringBuilder();
    }

    public static void main(String[] args) throws IOException {
        DatagramSocket socket = new DatagramSocket(PORT);
        System.out.println("UDP server started on " + PORT);

        Map<String, SaveJob> jobs = new HashMap<String, SaveJob>();
        byte[] buf = new byte[BUF];

        while (true) {
            try {
                DatagramPacket packet = new DatagramPacket(buf,
buf.length);
                socket.receive(packet);

                String text = new String(packet.getData(), 0,
packet.getLength());
                InetAddress address = packet.getAddress();
                int port = packet.getPort();
                String key = address.getHostAddress() + ":" + port;

                System.out.println("from " + key + " : " + text);

                String reply = handle(text, key, jobs);
                if (reply != null) {
                    byte[] out = reply.getBytes();
                    DatagramPacket answer = new DatagramPacket(out,
out.length, address, port);
                    socket.send(answer);
                }
            }
        }
    }
}

```

```

        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static String handle(String text, String key, Map<String,
SaveJob> jobs) {
    SaveJob job = jobs.get(key);

    if (job != null) {
        int p = text.indexOf("#end##");
        if (p >= 0) {
            job.content.append(text.substring(0, p));
            jobs.remove(key);
            try {
                FileWriter fw = new FileWriter(job.name);
                fw.write(job.content.toString());
                fw.close();
                return "OK: file " + job.name + " created";
            } catch (IOException e) {
                return "ERROR: cannot create file";
            }
        }
        job.content.append(text).append('\n');
        return null;
    }

    if (text.startsWith("save ")) {
        String name = text.substring(5).trim();
        if (name.isEmpty()) {
            return "ERROR: cannot create file";
        }
        SaveJob next = new SaveJob();
        next.name = name;
        jobs.put(key, next);
        return null;
    }

    return text;
}
}

```

Вывод: Изучил основы программирования сетевых приложений на базе библиотеки `java.net`; приобрёл навыки по практическому использованию классов `DatagramSocket` и `DatagramPacket` для реализации сетевых приложений на базе протокола UDP.